APEXQUANT
TERMINAL v10.0

INSTITUTIONAL QUANTITATIVE SPORTS TRADING & MULTI-API FAILOVER ARCHITECTURE

Zero-Hallucination OCR, Deterministic Math, 3D Glassmorphic UI, and Self-Learning Governance

Creator / Lead Architect

Muhammad Atif

Date

November 2026

Document ID: AQ-WP-10.0 | Analytical research tool. Not financial advice. No outcome is guaranteed.

Contents

#	Section	Scope
1	Executive Summary & Domain Scope	Tier-1 sports, purity filters, Multi-API Failover Shield
2	System Workflow Architecture	Ingestion, Vision OCR, math, research, memory vault, UI
3	Deterministic Formulas & Code	De-vig, DNB, Asian Handicap EV, Poisson, Kelly
4	Governance & Veto Rules	VR-01 to VR-04 and audit case study
5	Master Prompts & Agent Skills	Claude and Gemini prompts, chat ops, schedule scrapers
6	Source Code Appendix	config, schemas, engine, vault, router, main, CSS

Important notice. ApexQuant is an analytical decision-support system. Sports markets are efficient and uncertain; positive modeled expected value does not imply profit in any finite sample. Wagering carries risk of loss, is restricted in many jurisdictions, and should only be done legally, with money you can afford to lose. Nothing in this document is financial or betting advice.

Section 1: Executive Summary & Domain Scope

ApexQuant Terminal v10.0 is a deterministic quantitative analysis platform for regulated sports markets. Its design principle is simple: **language models may read and research, but only audited code may calculate and decide.** Every probability, edge and stake is produced by an offline mathematical engine; models are confined to extraction (OCR) and grounded qualitative notes, and are required to return null rather than guess.

1.1 Tier-1 Regulated Sports

Sport	Eligible competitions (purity filter)	Primary markets	Model basis
Football	Premier League, La Liga, Serie A, Bundesliga, Ligue 1, UCL, UEL	1X2, DNB, Asian Handicap, BTTS, totals	Bivariate Poisson goal model; sharp 1X2 de-vig
Cricket	ICC Test/ODI/T20I, IPL, BBL, PSL	Match winner, innings runs, top batter	Sharp-price de-vig; venue and toss adjustments
Tennis	ATP Tour, WTA Tour, Grand Slams	Match winner, set/game handicap, totals	Two-way de-vig; surface-adjusted hold rates
Basketball	NBA, EuroLeague	Spread, moneyline, totals	Two-way de-vig; pace and rest factors
UFC	Numbered events and Fight Nights	Moneyline, method, distance	Two-way de-vig; line-movement checks

1.2 Anti-Corruption Purity Filters

- **Competition whitelist:** any fixture outside the Tier-1 whitelist is rejected before pricing (reason code *PURITY*). Lower-division, friendly, exhibition and unregulated events are excluded because thin liquidity and integrity risk make prices unreliable.
- **Bookmaker blacklist:** known trap or soft-outlier books (for example OptionBet) are never used as price references and are rejected as bet sources (VR-03).
- **Sharp-price anchoring:** true probabilities derive only from sharp-book prices (Pinnacle, Betfair Exchange, Matchbook); recreational book prices are tested against that anchor.

- **Suspicious-movement flag:** large unexplained late moves against the model mark a fixture for manual review instead of automated action.

1.3 Multi-API Failover Shield

A single dependency should never halt analysis. Requests traverse an ordered chain; each hop has a timeout and bounded retry, and a failure simply advances to the next provider. The final tier is a 100% offline floor: the deterministic engine needs no network at all.

Tier	Provider	Role	On failure
1	Gemini (free API)	Vision OCR and grounded summarization	Advance to Tier 2
2	The Odds API	Structured multi-book odds feed	Advance to Tier 3
3	Groq Llama-3.3	Fast text reasoning and JSON extraction	Advance to Tier 4
4	DuckDuckGo	Keyless web grounding (news, lineups)	Advance to Tier 5
5	Offline fallback	Deterministic math on manually pasted odds	Never fails; qualitative notes marked unavailable

Section 2: Complete System Workflow Architecture

The platform is a linear pipeline of six nodes. Each node has a single responsibility, a typed contract (see *schemas.py*) and a defined failure mode.

Node	Module	Responsibility	Failure behaviour
N1 Ingestion	main.py, router	**Dual path:** (a) **Ctrl+V clipboard paste** of a bet-slip or odds screenshot; (b) **live Sharp auto-fetch** of sharp-book odds via the API chain.	Falls back to manual odds entry; never blocks.
N2 Vision OCR	multi_api_router.py	Extracts bookmaker, market, line and decimal odds as strict JSON. Prompt forbids inference: unreadable fields must be null. Output is schema-validated (odds > 1.0) and cross-checked against the live feed.	Validation failure rejects the field; user confirms manually.
N3 Offline Deterministic Math	quant_engine.py	De-vig, derived DNB, push-adjusted Asian Handicap EV, Poisson clean-sheet, quarter-Kelly. Pure functions, no I/O, no randomness.	None required: runs fully offline.
N4 Grounded Research	router (Tiers 1-4)	Lineups, injuries, weather and motivation notes, each tied to a retrieved source. Advisory only: it may trigger a veto but can never raise a stake.	Marked unavailable; math proceeds.
N5 Memory Vault	memory_vault.py	Append-only SQLite log of bets, vetoes and lessons; computes closing-line value (CLV). Lessons feed prompt context, not the math constants.	Logging errors surface; no silent loss.
N6 3D Glassmorphic UI	styles3d.css	Renders verdict cards (approved bet or veto reasons), CLV and bankroll. Respects reduced-motion preferences.	Plain-text rendering if CSS fails.

2.1 Data Flow

```
[Ctrl+V image | Sharp auto-fetch]
        |
        v
  N1 Ingestion --> N2 Vision OCR (JSON, nulls allowed) --> schema validation
        |
        v
  N3 Deterministic Math: de-vig -> fair prices -> EV (push-adjusted) -> quarter-Kelly
        |                                    ^
        v                                    |
  N4 Grounded Research (advisory) ----------+  (can only veto, never boost)
        |
        v
  Governance VR-01..VR-04  -->  Verdict { approved | vetoes[] }
        |
        v
  N5 Memory Vault (log, CLV, lessons)  -->  N6 3D UI render
```

2.2 Zero-Hallucination Controls

- Models never produce numbers that enter the stake calculation; they only transcribe values visible in the input.
- Every extracted field carries a null option; missing data produces a visible gap, not an invented value.
- Extracted odds are cross-validated against the API feed; mismatches beyond tolerance are flagged.
- Qualitative claims must cite a retrieved source or be discarded.

Section 3: Deterministic Mathematical Formulas & Code Implementation

All functions below are implemented in *quant_engine.py* (Appendix) and were executed offline before publication. Decimal odds O include the returned stake.

3.1 Multiplicative (Proportional) De-Vigging

A bookmaker's implied probabilities sum to the booking $R = \Sigma(1/O_j) > 1$; the excess is the margin. Proportional de-vigging assumes the margin is spread proportionally across outcomes.

$$P_{true,i} = (1 / O_i) / \Sigma_j (1 / O_j)$$

Proof of validity. Let $q_i = 1/O_i$ and $R = \Sigma q_j$. Then $\Sigma_i P_{true,i} = (\Sigma q_i)/R = 1$, and each $P_{true,i} \in (0,1)$, so the result is a valid probability distribution that preserves the ratios $q_i/q_j = O_j/O_i$ of the quoted prices. **Worked example:** 1X2 at 2.10 / 3.40 / 3.60 gives R = 1.0999; true probabilities are 0.4543 / 0.2806 / 0.2650.

```
def devig_multiplicative(odds):
    inv = [1.0 / o for o in odds]
    book = sum(inv)                  # overround R >= 1
    return [x / book for x in inv]
```

Limitation: proportional removal slightly overstates favourite-longshot bias; it is used because it is transparent and stable, and sharp books carry a small margin.

3.2 Derived Draw No Bet (Handicap 0) from Sharp 1X2

A draw refunds a DNB stake, so the market is the 1X2 distribution conditioned on no draw:

$$P_{DNB,H} = P_H / (P_H + P_A), \quad P_{DNB,A} = P_A / (P_H + P_A), \quad O_{fair} = 1 / P_{DNB}$$

Using the example above: $P_H/(P_H+P_A) = 0.6316$, fair DNB home price 1.583 and away 2.714. A quoted DNB price above fair is a candidate edge.

```
def derive_dnb(odds_1x2):
    ph, pd, pa = devig_multiplicative(odds_1x2)
    ph_dnb, pa_dnb = ph / (ph + pa), pa / (ph + pa)
    return {"p_home": ph_dnb, "p_away": pa_dnb,
            "fair_home": 1 / ph_dnb, "fair_away": 1 / pa_dnb, "p_draw_refund": pd}
```

3.3 Integer & Quarter Asian Handicap Push-Adjusted EV

For a decimal price *O* with win probability P_{win} and push probability P_{push} (stake returned):

$$EV = (P_{win} \times O) + (P_{push} \times 1.0) - 1$$

Derivation. Per unit staked: a win returns O, a push returns 1, a loss returns 0. E[return] = $P_{win}O + P_{push}\cdot 1 + P_{loss}\cdot 0$, and EV = E[return] − 1. Win, push and loss probabilities come from the goal-difference distribution $D = G_H - G_A$ under independent Poisson goals. For a home integer handicap *h*: win if D + h > 0, push if D + h = 0.

Quarter lines split the stake equally across the two adjacent lines: a −0.25 bet is 50% on handicap 0 and 50% on −0.5, so $EV = \tfrac{1}{2}EV_0 + \tfrac{1}{2}EV_{-0.5}$. **Example** ($\lambda_H = 1.55$, $\lambda_A = 1.10$): at handicap 0, $P_{win} = 0.477$, $P_{push} = 0.253$; at odds 1.95, EV = +18.3%. At −0.25 and odds 2.05, EV = +10.5%. These are model outputs from assumed λ inputs, not predictions of results.

```
def ev_push_adjusted(p_win, p_push, odds):
    return p_win * odds + p_push * 1.0 - 1.0

def asian_handicap_ev(dist, handicap, odds):
    twice = handicap * 2
    if abs(twice - round(twice)) < 1e-9:         # integer or half line
        legs = [(handicap, 1.0)]
    else:                                        # quarter line: split stake
        legs = [(handicap - 0.25, 0.5), (handicap + 0.25, 0.5)]
    ev = pw = pp = 0.0
    for line, w in legs:
        if abs(line - round(line)) < 1e-9:       # integer leg: push possible
            win, push, _ = _settle_integer(dist, int(round(line)))
        else:                                    # half leg: no push
            win = sum(p for d, p in dist.items() if d + line > 0); push = 0.0
        ev += w * ev_push_adjusted(win, push, odds); pw += w * win; pp += w * push
    return {"ev": ev, "p_win": pw, "p_push": pp, "p_lose": 1 - pw - pp}
```

3.4 Bivariate Poisson Clean-Sheet Modeling

Goals follow $G_H = X_1 + X_3$, $G_A = X_2 + X_3$ with independent Poisson $X_k(\lambda_k)$; X_3 is the shared (correlation) component. A 0-0 requires all three components to be zero:

$$P(0\text{-}0) = e^{-(\lambda_1+\lambda_2+\lambda_3)} = e^{-\lambda_{total}}$$

Proof. $P(G_H=0, G_A=0) = P(X_1=0)P(X_2=0)P(X_3=0) = e^{-\lambda_1}e^{-\lambda_2}e^{-\lambda_3}$. Because the three components are independent, the product collapses to a single exponential in the total rate. **Example:** $\lambda_{total} = 2.65$ gives P(0-0) = 7.07%. A team clean sheet is P(opponent scores 0) = $e^{-\lambda_{opp}}$.

```
def p_nil_nil(lam_total):   return math.exp(-lam_total)
def p_clean_sheet(lam_opp): return math.exp(-lam_opp)
```

3.5 Fractional Quarter-Kelly Criterion Staking

With b = O − 1, win probability p and loss probability q (pushes excluded from q), full Kelly maximizes expected log-growth:

$$f^* = (b\cdot p - q) / b, \quad stake = min(0.25 \times f^*, cap)$$

Why a quarter. Full Kelly is optimal only if p is known exactly. Model error in p is unavoidable, and Kelly staking on an overestimated p is aggressive; fractional Kelly sacrifices a little growth for a large reduction in variance and drawdown. The quarter factor plus a hard bankroll cap (VR-04) bounds the damage from a mis-specified model. Example: p = 0.55, O = 1.95 gives f* = 7.6% and a quarter-Kelly stake of 1.9% of bankroll.

```
def kelly_full(p_win, odds, p_push=0.0):
    b = odds - 1.0
    q = 1.0 - p_win - p_push
    return max(0.0, (b * p_win - q) / b)

def quarter_kelly(p_win, odds, p_push=0.0, fraction=0.25, cap=0.025):
    return min(fraction * kelly_full(p_win, odds, p_push), cap)
```

Section 4: Hardcoded Governance & Veto Rules (VR-01 to VR-04)

Governance rules are code, not prompts. They execute after the math and can only remove or shrink a bet; no rule can increase risk.

Rule	Name	Trigger	Action
VR-01	BTTS Team-Total Veto	Either team's Under 1.5 team-total price is ≤ 1.55	Veto any BTTS Yes selection
VR-02	One Match, One Bullet (Single-Sniper)	More than one qualifying selection in a match	Keep only the highest-EV selection; discard the rest
VR-03	Outlier & OptionBet Trap Rejection	Price deviates > 15% from sharp fair price, or source book is blacklisted	Reject the line; log reason
VR-04	Quarter-Kelly Capping	Any stake calculation	Stake = min(0.25 × f*, 2.5% of bankroll)

4.1 VR-01: BTTS Team-Total Veto

When the market prices a team's Under 1.5 goals at 1.55 or shorter (implied roughly 64% or more before margin), the match profile is low-scoring for that side. A BTTS Yes position is highly correlated with that team scoring at least once, so the two market views conflict. The rule resolves the conflict conservatively by vetoing BTTS Yes rather than trusting an independent-goals model on a market the books are signalling against.

4.2 VR-02: One Match, One Bullet (Single-Sniper Protocol)

Multiple selections within one match (for example a handicap, a total and a BTTS) are positively correlated; staking all of them is hidden over-exposure. The protocol permits exactly one selection per match: the qualifying candidate with the highest EV. Exposure per fixture is therefore bounded by one VR-04 stake.

4.3 VR-03: Outlier & OptionBet Trap Rejection

An apparent large edge against a sharp anchor is far more often a stale, erroneous or trap price than a true mispricing. Prices deviating by more than 15% from the sharp fair price are rejected, and blacklisted books are excluded outright. The rule also logs each rejection to the Memory Vault so persistent offenders can be reviewed.

4.4 VR-04: Quarter-Kelly Capping

Every stake is the lesser of 25% of full Kelly and 2.5% of current bankroll. Candidates with EV below the minimum edge (2%) are not staked at all.

4.5 Historical Audit Case Study: Shakhtar vs Sporting (0:1)

Scope note: the final score 0:1 is the audit reference supplied by the platform owner. The pre-match prices and probabilities below are an **illustrative reconstruction** to demonstrate rule behaviour and should be replaced with the logged Memory Vault values before external distribution.

Step	Illustrative reading	Governance outcome
Candidates	Shakhtar AH 0 and BTTS Yes both showed positive modeled EV	Both enter governance
Team-total check	Shakhtar Under 1.5 priced at or below 1.55 (low-scoring home profile)	VR-01 vetoes BTTS Yes
Single-sniper	Only AH selection remains; no second bullet permitted	VR-02 passes one bet
Staking	Quarter-Kelly stake under the 2.5% cap	VR-04 caps exposure
Result 0:1	Under the scoreline, BTTS Yes (vetoed) loses; the capped handicap position loses a small, bounded stake	Veto avoided the correlated loss; one capped loss recorded

Lessons logged: (1) low team-total pricing is informative evidence against BTTS Yes, which motivated the VR-01 threshold; (2) a single match result cannot validate a rule, so the Vault evaluates rules by CLV and drawdown over large samples rather than individual outcomes.

Section 5: Master Prompts & AI Agent Skills Suite

Prompts are copy-pasteable and un-truncated. Each begins with a shared **Global Contract** which must be pasted first in every session. Paste a prompt into Claude or Gemini; both variants are given because their strengths differ (Claude for multi-file code with tests; Gemini for vision and long-context reading).

5.0 Global Contract

```
GLOBAL CONTRACT (paste first, applies to every prompt below)
You are a senior quantitative engineer on ApexQuant Terminal v10.0.
Rules: (1) Never invent odds, scores, lineups or statistics. If a value is not in the
input or a cited source, output null. (2) Every number used for staking must come from
deterministic code, never from model text. (3) Return only what is asked, in the stated
format. (4) State assumptions explicitly. (5) Include unit tests for all math. (6) Do not
claim profit or certainty; this is analysis, not advice.
```

Phase 1: Core Math Engine

Claude prompt

```
Build quant_engine.py in pure Python 3.11 (standard library only, no network).
Implement and unit-test (pytest):
  1. devig_multiplicative(odds): P_i = (1/O_i)/sum(1/O_j)
  2. derive_dnb(odds_1x2) from the de-vigged 1X2
  3. goal_diff_distribution(lam_h, lam_a) with independent Poisson, max 15 goals
  4. asian_handicap_ev(dist, handicap, odds) supporting integer, half and quarter lines,
     with EV = P_win*O + P_push*1 - 1; quarter lines split stake across adjacent lines
  5. p_nil_nil(lam_total) = exp(-lam_total)
  6. kelly_full and quarter_kelly(fraction=0.25, cap=0.025)
Tests must include: probabilities sum to 1; DNB home+away = 1; quarter-line EV equals the
average of its two legs; kelly returns 0 for negative edge. Show the pytest output.
```

Gemini prompt

```
Role: quantitative engineer. Write quant_engine.py (Python 3.11, stdlib only, deterministic,
no network, no randomness) containing: devig_multiplicative, derive_dnb,
goal_diff_distribution (independent Poisson, truncate at 15 goals), asian_handicap_ev
(integer/half/quarter lines; quarter = 50/50 split of adjacent lines; EV = P_win*O + P_push - 1),
p_nil_nil = exp(-lam_total), kelly_full, quarter_kelly (0.25 fraction, 2.5% cap).
Then write test_quant_engine.py with pytest covering normalization (sums to 1), DNB conditional
identity, quarter-line averaging, zero Kelly on negative edge. Explain each formula in one line
in a docstring. Output two code blocks only.
```

Phase 2: Schemas, Config & Multi-API Router

Claude prompt

```
Create config.py, schemas.py (pydantic v2) and multi_api_router.py.
Router chain in strict order: Gemini -> The Odds API -> Groq (llama-3.3-70b-versatile) ->
DuckDuckGo -> offline floor. Requirements: httpx with 8s timeout, one retry per provider,
failures advance to the next provider, record per-provider failure counts, and the offline
floor must always return a ProviderResult(ok=True). API keys only from environment
variables, never hard-coded or logged. Odds schema: decimal odds must be > 1.0. Provide
tests that mock every provider failing and assert the offline floor result.
```

Gemini prompt

```
Write config.py, schemas.py (pydantic v2: OddsLine, MatchSnapshot, Candidate, Verdict,
ProviderResult) and multi_api_router.py. Implement an ordered failover chain
[Gemini, The Odds API, Groq llama-3.3-70b-versatile, DuckDuckGo, offline] using httpx
(timeout 8s, 1 retry). Read keys from env vars GEMINI_API_KEY, ODDS_API_KEY, GROQ_API_KEY.
Never log keys. Validate odds > 1.0. Add pytest tests using respx or monkeypatch where all
network providers raise, proving the offline result is returned. Output code blocks only.
```

Phase 3: Vision OCR, Research & Memory Vault

Claude prompt

```
Create the ingestion layer and memory_vault.py.
OCR prompt contract: extract bookmaker, market, selection, line, decimal odds from a pasted
screenshot as strict JSON; every unreadable field must be null; no guessing; include a
confidence value per field. Validate against schemas.py and cross-check odds against the
live feed (flag mismatches > 5%). Research module: only return facts with a cited URL.
memory_vault.py: SQLite tables bets, vetoes, lessons; settle(); clv_report() computing
mean(odds/closing_odds - 1); recent_lessons(). Lessons are text context only and must never
modify math constants. Provide tests for CLV on a toy dataset.
```

Gemini prompt

```
Build (a) an OCR extraction function that sends a screenshot to Gemini vision with this
system instruction: "Extract bookmaker, market, selection, line and decimal odds. Return JSON
only. Use null for anything not clearly visible. Never infer or round." plus a per-field
confidence; (b) a pydantic validator rejecting odds <= 1.0; (c) memory_vault.py with SQLite
tables bets, vetoes, lessons and a clv_report(); (d) pytest tests with a fixture image-less
mock. Explain in 5 lines how nulls propagate so the UI shows gaps instead of invented values.
```

Phase 4: Governance, FastAPI & 3D UI

Claude prompt

```
Create main.py (FastAPI) and styles3d.css.
Endpoints: POST /analyze (MatchSnapshot + lam_home + lam_away -> Verdict), POST /ingest/image,
GET /vault/clv. Governance after the math: VR-01 veto BTTS Yes if either team Under 1.5
<= 1.55; VR-02 keep only the highest-EV candidate per match; VR-03 reject prices deviating
> 15% from the sharp fair price and any blacklisted bookmaker; VR-04 stake =
min(0.25*Kelly, 2.5% bankroll) and skip EV < 2%. Purity filter rejects non Tier-1
competitions. Log every veto and bet to the vault. CSS: dark navy #0F172A, emerald #10B981,
glassmorphic cards with backdrop-filter, 3D hover tilt, prefers-reduced-motion support.
```

Gemini prompt

```
Write main.py (FastAPI) and styles3d.css for ApexQuant v10. Implement /analyze, /ingest/image
and /vault/clv. After computing candidates with quant_engine, apply: VR-01 (veto BTTS Yes when
either Under 1.5 team-total <= 1.55), VR-02 (one selection per match, highest EV),
VR-03 (reject > 15% deviation from sharp price or blacklisted book), VR-04 (stake =
min(0.25*f*, 2.5% bankroll); skip EV < 2%). Reject non Tier-1 competitions first. Log vetoes.
CSS: #0F172A background, #10B981 accent, glass cards (backdrop-filter blur), perspective hover
tilt, 3D pressable button, reduced-motion fallback. Provide pytest tests for each rule.
```

5.5 Chat Operational Prompts

Pre-match analysis

```
Using ONLY the data below, run the ApexQuant analysis. Do not add odds or stats that are not
provided. Steps: (1) list inputs you received; (2) list any missing fields as null; (3) call the
deterministic engine outputs I supply (fair prices, EV, stake) and explain them; (4) apply VR-01
to VR-04 and state which rules fired and why; (5) give one verdict: APPROVED (single selection)
or NO BET, with reasons. Do not predict the result. Data: <paste sharp 1X2, AH lines, team totals>
```

Screenshot OCR check

```
Read this screenshot. Return JSON only: {"bookmaker":..,"market":..,"selection":..,"line":..,
"odds":..,"confidence":{..}}. Use null for any field not clearly legible. Never guess digits.
After the JSON, list the fields you are least sure of.
```

Post-match audit

```
Match: <teams, final score>. Bet logged: <market, line, odds, stake%>. Closing odds: <value>.
Compute CLV = odds/closing - 1. State whether any VR rule should have fired and did not.
Distinguish a bad bet from bad luck (a single result is not evidence). Propose at most one
lesson as a testable hypothesis, not a rule change.
```

Veto explainer

```
Explain in plain language why <selection> was vetoed under <VR-xx>, in under 120 words, and
state what price or data change would have allowed it. Do not suggest ways to bypass the rule.
```

5.6 Daily Tier-1 Schedule Scraper Prompts

Use with a search-enabled model. Both prompts require source URLs so listings can be verified, and tell the model to omit rather than guess.

Cricket (daily)

```
Search the web for today's date's fixtures in Tier-1 cricket ONLY: ICC Test, ODI and T20I
internationals, IPL, BBL, PSL. For each fixture return a table: competition, team A vs team B,
venue, start time in UTC and in Asia/Karachi, format, and a source URL. Exclude domestic
second-tier, club-level and unregulated leagues. If a start time is unconfirmed write
"unconfirmed". Do not guess. End with the number of fixtures found and the sources used.
```

Football (daily)

```
Search the web for today's date's fixtures in Tier-1 football ONLY: Premier League, La Liga,
Serie A, Bundesliga, Ligue 1, UEFA Champions League, UEFA Europa League. For each fixture
return a table: competition, home vs away, venue, kickoff in UTC and in Asia/Karachi,
confirmed lineups if published (else "not yet published"), and a source URL. Exclude
friendlies and lower divisions. Do not infer lineups or injuries; cite each claim or omit it.
```

Section 6: Complete Python Source Code Appendix

Listings are the exact files compiled into this document. The pure-math module *quant_engine.py* was executed during the build; the remaining modules passed a Python syntax check. Network-dependent code (*multi_api_router.py*) requires your own API keys and has not been exercised against live services here, and third-party endpoints and model names may change, so verify them against provider documentation. Install: *pip install fastapi uvicorn pydantic httpx*. Run: *uvicorn main:app --reload*.

A.1 config.py

```
"""config.py - central configuration for ApexQuant Terminal v10.0."""
import os
from dataclasses import dataclass, field

TIER1_SPORTS = ("football", "cricket", "tennis", "basketball", "ufc")

# Anti-corruption purity filter: only regulated, Tier-1 competitions.
TIER1_COMPETITIONS = {
    "football": {"Premier League", "La Liga", "Serie A", "Bundesliga",
                 "Ligue 1", "UEFA Champions League", "UEFA Europa League"},
    "cricket": {"ICC Test", "ICC ODI", "ICC T20I", "IPL", "BBL", "PSL"},
    "tennis": {"ATP Tour", "WTA Tour", "Grand Slam"},
    "basketball": {"NBA", "EuroLeague"},
    "ufc": {"UFC Numbered", "UFC Fight Night"},
}
BLOCKED_BOOKMAKERS = {"optionbet"}          # VR-03 trap list
SHARP_BOOKMAKERS = ("pinnacle", "betfair_ex_eu", "matchbook")


@dataclass(frozen=True)
class Settings:
    gemini_key: str = os.getenv("GEMINI_API_KEY", "")
    odds_api_key: str = os.getenv("ODDS_API_KEY", "")
    groq_key: str = os.getenv("GROQ_API_KEY", "")
    gemini_model: str = os.getenv("GEMINI_MODEL", "gemini-2.0-flash")
    groq_model: str = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")
    http_timeout_s: float = 8.0
    retries_per_provider: int = 1
    bankroll: float = float(os.getenv("APEX_BANKROLL", "1000"))
    kelly_fraction: float = 0.25            # quarter-Kelly
    max_stake_pct: float = 0.025            # VR-04 hard cap (2.5% of bankroll)
    min_edge: float = 0.02                  # minimum EV to qualify
    btts_under15_veto: float = 1.55         # VR-01 threshold
    outlier_dev_pct: float = 0.15           # VR-03 deviation vs sharp consensus
    vault_path: str = os.getenv("APEX_VAULT", "memory_vault.sqlite3")


SETTINGS = Settings()
```

A.2 schemas.py

```
"""schemas.py - typed data contracts (pydantic v2)."""
from __future__ import annotations
from enum import Enum
from typing import List, Optional
from pydantic import BaseModel, Field, field_validator


class Market(str, Enum):
    ML_1X2 = "1X2"
    DNB = "DNB"
    AH = "AH"
    BTTS_YES = "BTTS_YES"
    TEAM_TOTAL_U15 = "TEAM_TOTAL_U15"
    TOTAL_OVER = "TOTAL_OVER"
    TOTAL_UNDER = "TOTAL_UNDER"


class OddsLine(BaseModel):
    bookmaker: str
    market: Market
    selection: str
    odds: float = Field(gt=1.0, description="Decimal odds")
    line: Optional[float] = None            # handicap / total line

    @field_validator("bookmaker")
    @classmethod
    def _norm(cls, v: str) -> str:
        return v.strip().lower()


class MatchSnapshot(BaseModel):
    match_id: str
    sport: str
    competition: str
    home: str
    away: str
    lines: List[OddsLine]
    team_total_u15_home: Optional[float] = None
    team_total_u15_away: Optional[float] = None


class Candidate(BaseModel):
    match_id: str
    market: Market
    selection: str
    line: Optional[float]
    odds: float
    p_win: float
    p_push: float = 0.0
    ev: float
    kelly_full: float
    stake_pct: float
    bookmaker: str


class Verdict(BaseModel):
    match_id: str
    approved: Optional[Candidate] = None
    vetoes: List[str] = []                  # e.g. ["VR-01: ..."]
    notes: List[str] = []


class ProviderResult(BaseModel):
    provider: str
    ok: bool
    payload: Optional[dict] = None
    error: Optional[str] = None
    latency_ms: Optional[float] = None
```

A.3 quant_engine.py

```
"""quant_engine.py - deterministic, offline mathematics. No network, no LLM."""
from __future__ import annotations
import math
from typing import Dict, List, Sequence, Tuple


# ---------- 1. Multiplicative (proportional) de-vig ----------------------
def implied(odds: float) -> float:
    return 1.0 / odds


def devig_multiplicative(odds: Sequence[float]) -> List[float]:
    """P_true_i = (1/O_i) / sum_j(1/O_j)."""
    inv = [implied(o) for o in odds]
    book = sum(inv)                          # overround R >= 1
    return [x / book for x in inv]


def overround(odds: Sequence[float]) -> float:
    return sum(implied(o) for o in odds) - 1.0


# ---------- 2. Draw No Bet derived from sharp 1X2 ----------------------
def derive_dnb(odds_1x2: Tuple[float, float, float]) -> Dict[str, float]:
    """Handicap 0: a draw refunds the stake, so condition on no-draw."""
    ph, pd, pa = devig_multiplicative(odds_1x2)
    ph_dnb, pa_dnb = ph / (ph + pa), pa / (ph + pa)
    return {"p_home": ph_dnb, "p_away": pa_dnb,
            "fair_home": 1 / ph_dnb, "fair_away": 1 / pa_dnb, "p_draw_refund": pd}


# ---------- 3. Asian Handicap push-adjusted EV --------------------------
def poisson_pmf(k: int, lam: float) -> float:
    return math.exp(-lam) * lam ** k / math.factorial(k)


def goal_diff_distribution(lam_h: float, lam_a: float, max_goals: int = 15) -> Dict[int, float]:
    dist: Dict[int, float] = {}
    for h in range(max_goals + 1):
        ph = poisson_pmf(h, lam_h)
        for a in range(max_goals + 1):
            dist[h - a] = dist.get(h - a, 0.0) + ph * poisson_pmf(a, lam_a)
    return dist


def _settle_integer(dist: Dict[int, float], handicap: int) -> Tuple[float, float, float]:
    """Home bet at integer handicap. Returns (p_win, p_push, p_lose)."""
    win = sum(p for d, p in dist.items() if d + handicap > 0)
    push = sum(p for d, p in dist.items() if d + handicap == 0)
    return win, push, 1.0 - win - push


def ev_push_adjusted(p_win: float, p_push: float, odds: float) -> float:
    """EV = P_win * O + P_push * 1.0 - 1   (decimal odds, stake returned on push)."""
    return p_win * odds + p_push * 1.0 - 1.0


def asian_handicap_ev(dist: Dict[int, float], handicap: float, odds: float) -> Dict[str, float]:
    """Integer lines settle directly; quarter lines split stake across the two
    adjacent half-unit lines (e.g. -0.25 = 50% on 0 and 50% on -0.5)."""
    twice = handicap * 2
    if abs(twice - round(twice)) < 1e-9:     # integer or half line
        legs = [(handicap, 1.0)]
    else:                                    # quarter line
        legs = [(handicap - 0.25, 0.5), (handicap + 0.25, 0.5)]
    ev = pw = pp = 0.0
    for line, w in legs:
        if abs(line - round(line)) < 1e-9:   # integer leg: push possible
            win, push, _ = _settle_integer(dist, int(round(line)))
        else:                                # half leg: no push
            win = sum(p for d, p in dist.items() if d + line > 0)
            push = 0.0
        ev += w * ev_push_adjusted(win, push, odds)
        pw += w * win
        pp += w * push
    return {"ev": ev, "p_win": pw, "p_push": pp, "p_lose": 1 - pw - pp}


# ---------- 4. Bivariate Poisson clean sheet ----------------------------
def p_nil_nil(lam_total: float) -> float:
    """P(0-0) = exp(-lambda_total)."""
```

```
    return math.exp(-lam_total)


def p_clean_sheet(lam_opp: float) -> float:
    """P(team keeps a clean sheet) = P(opponent scores 0) = exp(-lambda_opp)."""
    return math.exp(-lam_opp)


# ---------- 5. Fractional Kelly ----------------------------------------
def kelly_full(p_win: float, odds: float, p_push: float = 0.0) -> float:
    b = odds - 1.0
    q = 1.0 - p_win - p_push
    f = (b * p_win - q) / b
    return max(0.0, f)


def quarter_kelly(p_win: float, odds: float, p_push: float = 0.0,
                  fraction: float = 0.25, cap: float = 0.025) -> float:
    return min(fraction * kelly_full(p_win, odds, p_push), cap)


if __name__ == "__main__":
    print([round(x, 4) for x in devig_multiplicative([2.10, 3.40, 3.60])])
    print(derive_dnb((2.10, 3.40, 3.60)))
    d = goal_diff_distribution(1.55, 1.10)
    print(asian_handicap_ev(d, 0, 1.95), asian_handicap_ev(d, -0.25, 2.05))
    print(p_nil_nil(2.65), quarter_kelly(0.55, 1.95))
```

A.4 memory_vault.py

```
"""memory_vault.py - self-learning governance memory (SQLite, append-only audit log)."""
import json
import sqlite3
import time
from typing import List, Optional

from config import SETTINGS

_SCHEMA = """
CREATE TABLE IF NOT EXISTS bets(
  id INTEGER PRIMARY KEY AUTOINCREMENT, ts REAL, match_id TEXT, market TEXT,
  selection TEXT, line REAL, odds REAL, p_win REAL, ev REAL, stake_pct REAL,
  result TEXT DEFAULT NULL, closing_odds REAL DEFAULT NULL, note TEXT);
CREATE TABLE IF NOT EXISTS vetoes(
  id INTEGER PRIMARY KEY AUTOINCREMENT, ts REAL, match_id TEXT, rule TEXT, detail TEXT);
CREATE TABLE IF NOT EXISTS lessons(
  id INTEGER PRIMARY KEY AUTOINCREMENT, ts REAL, rule TEXT, lesson TEXT);
"""


class MemoryVault:
    def __init__(self, path: str = SETTINGS.vault_path):
        self.db = sqlite3.connect(path)
        self.db.executescript(_SCHEMA)

    def log_bet(self, c, note: str = "") -> int:
        cur = self.db.execute(
            "INSERT INTO bets(ts,match_id,market,selection,line,odds,p_win,ev,stake_pct,note)"
            " VALUES(?,?,?,?,?,?,?,?,?,?)",
            (time.time(), c.match_id, c.market.value, c.selection, c.line,
             c.odds, c.p_win, c.ev, c.stake_pct, note))
        self.db.commit()
        return cur.lastrowid

    def log_veto(self, match_id: str, rule: str, detail: str) -> None:
        self.db.execute("INSERT INTO vetoes(ts,match_id,rule,detail) VALUES(?,?,?,?)",
                        (time.time(), match_id, rule, detail))
        self.db.commit()

    def settle(self, bet_id: int, result: str, closing_odds: Optional[float] = None) -> None:
        self.db.execute("UPDATE bets SET result=?, closing_odds=? WHERE id=?",
                        (result, closing_odds, bet_id))
        self.db.commit()

    def add_lesson(self, rule: str, lesson: str) -> None:
        self.db.execute("INSERT INTO lessons(ts,rule,lesson) VALUES(?,?,?)",
                        (time.time(), rule, lesson))
        self.db.commit()

    def clv_report(self) -> dict:
        """Closing-line value: the honest long-run skill metric."""
        rows = self.db.execute(
```

```
            "SELECT odds, closing_odds FROM bets WHERE closing_odds IS NOT NULL").fetchall()
        if not rows:
            return {"n": 0, "mean_clv": None}
        clv = [(o / c) - 1.0 for o, c in rows]
        return {"n": len(clv), "mean_clv": sum(clv) / len(clv)}

    def recent_lessons(self, n: int = 20) -> List[str]:
        return [r[0] for r in self.db.execute(
            "SELECT lesson FROM lessons ORDER BY id DESC LIMIT ?", (n,))]

    def export_json(self) -> str:
        rows = self.db.execute("SELECT * FROM bets").fetchall()
        return json.dumps(rows)
```

A.5 multi_api_router.py

```
"""multi_api_router.py - ordered multi-provider failover with a 100% offline floor."""
import time
from typing import Callable, Dict, List

import httpx

from config import SETTINGS
from schemas import ProviderResult


class Router:
    """Order: Gemini -> The Odds API -> Groq Llama-3.3 -> DuckDuckGo -> Offline."""

    def __init__(self):
        self.client = httpx.Client(timeout=SETTINGS.http_timeout_s)
        self.chain: List[Callable[[Dict], dict]] = [
            self._gemini, self._odds_api, self._groq, self._duckduckgo]
        self.health: Dict[str, int] = {}

    # --- providers ----------------------------------------------------
    def _gemini(self, req: Dict) -> dict:
        if not SETTINGS.gemini_key:
            raise RuntimeError("no GEMINI_API_KEY")
        url = ("https://generativelanguage.googleapis.com/v1beta/models/"
               f"{SETTINGS.gemini_model}:generateContent")
        body = {"contents": [{"parts": [{"text": req["prompt"]}]}],
                "generationConfig": {"temperature": 0.0}}
        r = self.client.post(url, params={"key": SETTINGS.gemini_key}, json=body)
        r.raise_for_status()
        return {"text": r.json()["candidates"][0]["content"]["parts"][0]["text"]}

    def _odds_api(self, req: Dict) -> dict:
        if not SETTINGS.odds_api_key or "sport_key" not in req:
            raise RuntimeError("odds request not applicable")
        url = f"https://api.the-odds-api.com/v4/sports/{req['sport_key']}/odds"
        r = self.client.get(url, params={"apiKey": SETTINGS.odds_api_key,
                                         "regions": "eu,uk", "markets": "h2h,spreads,totals",
                                         "oddsFormat": "decimal"})
        r.raise_for_status()
        return {"odds": r.json()}

    def _groq(self, req: Dict) -> dict:
        if not SETTINGS.groq_key:
            raise RuntimeError("no GROQ_API_KEY")
        r = self.client.post(
            "https://api.groq.com/openai/v1/chat/completions",
            headers={"Authorization": f"Bearer {SETTINGS.groq_key}"},
            json={"model": SETTINGS.groq_model, "temperature": 0,
                  "messages": [{"role": "user", "content": req["prompt"]}]})
        r.raise_for_status()
        return {"text": r.json()["choices"][0]["message"]["content"]}

    def _duckduckgo(self, req: Dict) -> dict:
        r = self.client.get("https://api.duckduckgo.com/",
                            params={"q": req["prompt"][:300], "format": "json",
                                    "no_html": 1, "skip_disambig": 1})
        r.raise_for_status()
        return {"text": r.json().get("AbstractText", ""), "grounding": "duckduckgo"}

    # --- offline floor ------------------------------------------------
    @staticmethod
    def _offline(req: Dict) -> dict:
        """Deterministic math needs no network; qualitative notes are marked unavailable."""
        return {"text": "", "offline": True,
                "note": "Qualitative research unavailable; deterministic engine only."}
```

```
    # --- orchestration ----------------------------------------------------
    def call(self, req: Dict) -> ProviderResult:
        for fn in self.chain:
            name = fn.__name__.lstrip("_")
            for _ in range(1 + SETTINGS.retries_per_provider):
                t0 = time.perf_counter()
                try:
                    payload = fn(req)
                    self.health[name] = 0
                    return ProviderResult(provider=name, ok=True, payload=payload,
                                          latency_ms=(time.perf_counter() - t0) * 1000)
                except Exception as exc:         # noqa: BLE001 - failover by design
                    self.health[name] = self.health.get(name, 0) + 1
                    last = f"{type(exc).__name__}: {exc}"
        return ProviderResult(provider="offline", ok=True, payload=self._offline(req),
                              error=last)
```

A.6 main.py

```
"""main.py - FastAPI entrypoint: ingestion -> math -> governance -> response."""
from typing import List

from fastapi import FastAPI, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware

import quant_engine as qe
from config import SETTINGS, TIER1_COMPETITIONS, BLOCKED_BOOKMAKERS
from memory_vault import MemoryVault
from multi_api_router import Router
from schemas import Candidate, Market, MatchSnapshot, Verdict

app = FastAPI(title="ApexQuant Terminal v10.0")
app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:8000"],
                   allow_methods=["*"], allow_headers=["*"])
vault, router = MemoryVault(), Router()


def purity_ok(m: MatchSnapshot) -> bool:
    return m.competition in TIER1_COMPETITIONS.get(m.sport, set())


def build_candidates(m: MatchSnapshot, lam_h: float, lam_a: float) -> List[Candidate]:
    dist = qe.goal_diff_distribution(lam_h, lam_a)
    out: List[Candidate] = []
    for ln in m.lines:
        if ln.market is not Market.AH or ln.line is None or ln.selection != "home":
            continue
        r = qe.asian_handicap_ev(dist, ln.line, ln.odds)
        stake = qe.quarter_kelly(r["p_win"], ln.odds, r["p_push"],
                                 SETTINGS.kelly_fraction, SETTINGS.max_stake_pct)
        out.append(Candidate(match_id=m.match_id, market=ln.market, selection=ln.selection,
                             line=ln.line, odds=ln.odds, p_win=r["p_win"], p_push=r["p_push"],
                             ev=r["ev"], kelly_full=qe.kelly_full(r["p_win"], ln.odds, r["p_push"]),
                             stake_pct=stake, bookmaker=ln.bookmaker))
    return out


def govern(m: MatchSnapshot, cands: List[Candidate], sharp_ref: dict) -> Verdict:
    v = Verdict(match_id=m.match_id)
    kept = []
    for c in cands:
        if c.bookmaker in BLOCKED_BOOKMAKERS:                              # VR-03
            v.vetoes.append(f"VR-03: blocked bookmaker {c.bookmaker}")
            continue
        ref = sharp_ref.get((c.market, c.line))
        if ref and abs(c.odds / ref - 1) > SETTINGS.outlier_dev_pct:         # VR-03
            v.vetoes.append(f"VR-03: outlier price {c.odds} vs sharp {ref:.2f}")
            continue
        if c.market is Market.BTTS_YES and any(
                x is not None and x <= SETTINGS.btts_under15_veto
                for x in (m.team_total_u15_home, m.team_total_u15_away)):    # VR-01
            v.vetoes.append("VR-01: BTTS vetoed, team Under 1.5 <= 1.55")
            continue
        if c.ev >= SETTINGS.min_edge and c.stake_pct > 0:
            kept.append(c)
    if kept:                                                                 # VR-02
        v.approved = max(kept, key=lambda c: c.ev)
        v.notes.append("VR-02: single sniper bet selected; all others discarded")
    return v


@app.post("/analyze")
```

```
def analyze(m: MatchSnapshot, lam_home: float, lam_away: float):
    if not purity_ok(m):
        return Verdict(match_id=m.match_id, vetoes=["PURITY: non Tier-1 competition"])
    sharp = {}   # populate from sharp-book fair prices (see Section 3)
    verdict = govern(m, build_candidates(m, lam_home, lam_away), sharp)
    for veto in verdict.vetoes:
        vault.log_veto(m.match_id, veto.split(":")[0], veto)
    if verdict.approved:
        vault.log_bet(verdict.approved)
    return verdict


@app.post("/ingest/image")
async def ingest_image(file: UploadFile = File(...)):
    data = await file.read()
    import base64
    res = router.call({"prompt": "Extract bookmaker, market, line, decimal odds as JSON only. "
                                 "Return null for any unreadable field. Do not guess.",
                       "image_b64": base64.b64encode(data).decode()})
    return res


@app.get("/vault/clv")
def clv():
    return vault.clv_report()
```

A.7 styles3d.css

```
/* styles3d.css - 3D glassmorphic terminal theme */
:root{
  --bg:#0F172A; --panel:rgba(30,41,59,.55); --edge:rgba(255,255,255,.10);
  --emerald:#10B981; --text:#E2E8F0; --muted:#94A3B8; --danger:#F43F5E;
  --mono:"JetBrains Mono","DejaVu Sans Mono",ui-monospace,monospace;
}
*{box-sizing:border-box}
body{margin:0;background:radial-gradient(1200px 600px at 20% -10%,#1E293B,#0F172A 60%);
  color:var(--text);font:14px/1.5 Inter,system-ui,sans-serif;min-height:100vh}
.scene{perspective:1400px;padding:32px}
.card{position:relative;background:var(--panel);border:1px solid var(--edge);
  border-radius:18px;padding:20px;backdrop-filter:blur(18px) saturate(140%);
  -webkit-backdrop-filter:blur(18px) saturate(140%);
  box-shadow:0 20px 50px rgba(0,0,0,.45),inset 0 1px 0 rgba(255,255,255,.08);
  transform-style:preserve-3d;transition:transform .35s ease,box-shadow .35s ease}
.card:hover{transform:rotateX(4deg) rotateY(-5deg) translateZ(18px);
  box-shadow:0 34px 70px rgba(16,185,129,.18),inset 0 1px 0 rgba(255,255,255,.12)}
.card h3{margin:0 0 8px;font-size:12px;letter-spacing:.14em;text-transform:uppercase;color:var(--muted)}
.metric{font:700 28px var(--mono);color:var(--emerald)}
.veto{border-color:rgba(244,63,94,.5)}
.veto .metric{color:var(--danger)}
.btn{background:linear-gradient(180deg,#10B981,#059669);color:#04130D;border:0;
  border-radius:12px;padding:10px 18px;font-weight:700;cursor:pointer;
  box-shadow:0 6px 0 #047857,0 12px 20px rgba(0,0,0,.35);transition:transform .08s}
.btn:active{transform:translateY(4px);box-shadow:0 2px 0 #047857}
@media (prefers-reduced-motion:reduce){.card,.btn{transition:none}.card:hover{transform:none}}
```

End of document. ApexQuant Terminal v10.0 | Muhammad Atif | November 2026